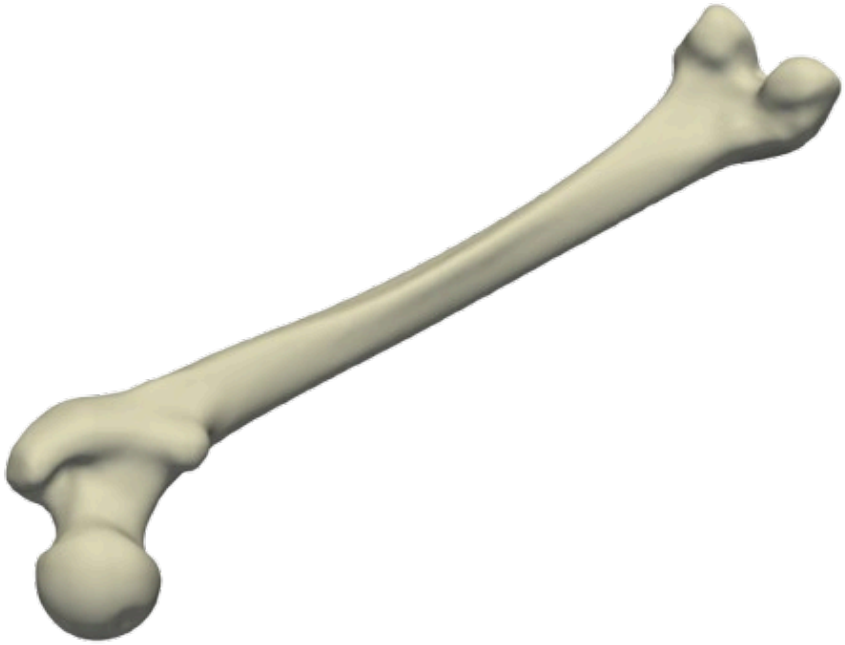
Statistical and Neural approaches in 3D Shape Modeling



Grenoble INP : ENSIMAG
Master's in Applied Mathematics
Modeling Project in C++ - Marek Bucki
Spring 2026

Martin Lainé, Basile Mouret, Timothé Boyer, Malik Hacini
27.01.2026 -

Abstract

For this project, we developed a comprehensive statistical shape analysis system for femur bones. Our approach relies on two complementary methodological pillars: Principal Component Analysis (PCA), enabling linear dimensionality reduction and identification of the main variation modes, and artificial neural networks, providing the capability to capture non-linear relationships in the data. Each femur in our dataset is represented as a 3D mesh comprising 18,291 vertices, resulting in a vector of dimension 54,873 in the shape space. The main objective of this work is to build a statistical shape model capable of reducing the dimensionality of the data using an autoencoder built with neural networks, generating new anatomically plausible femur shapes, PLACEHOLDER PCA.

Introduction

1 Motivation

2 Methods

2.1 Neural Networks

Neural Networks are a class of machine learning models inspired by the structure and function of biological neural networks. They are particularly well-suited for modeling complex, non-linear relationships in data. In this section, we describe the architecture and training process of the neural network implemented for our shape analysis task.

2.1.1 Modelisation of Neurons

A single artificial neuron receives an input x of size n, $x \in \mathbb{R}^n$. Each component associated with a weight. The neuron computes a weighted sum of these inputs and adds a bias term. Mathematically, this can be expressed as:

$$f(x) = b + \sum_{k=1}^{n} w_k x_k \quad x = (x_1 \ \dots \ x_n) \tag{1}$$

However, this result is just a linear combination of the inputs. To introduce non-linearity into the model, an activation function is applied to this weighted sum.

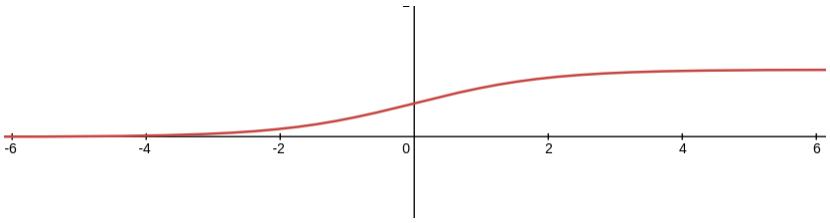


Figure 1 - Sigmoid activation function $\Phi(x) = \frac{1}{1+e^{-x}}$

Remark: The sigmoid function is one of many activation functions used in neural networks. Others include ReLU (Rectified Linear Unit), Tanh, or Softmax, each with its own advantages and applications.

Finally, the output of the neuron is given by the activation function applied to the weighted sum:

$$(\Phi \circ f)(x) = \Phi(f(x)) \tag{2}$$

PLACEHOLDER for Neuron Diagram
`auto` 2 - Structure of an Artificial Neuron

2.1.2 Layers and Network Architecture

Artificial neurons are organized into layers to form a neural network. A typical feedforward neural network consists of an input layer, one or more hidden layers, and an output layer. Each layer contains multiple neurons, and the output of one layer serves as the input to the next layer.

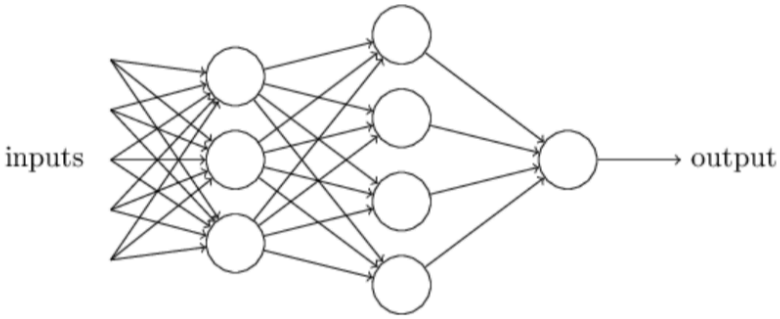


Figure 3 - Structure of a Neural Network

However, the weights and biases of the neurons are initially set to random values and need to be optimized through a training process.

2.1.3 Backpropagation and Training

Training a neural network involves adjusting the weights and biases to minimize the difference between the predicted outputs and the actual target values. This is typically done using a method called backpropagation combined with an optimization algorithm like gradient descent.

The training process consists of the following steps:

1. **`Forward Pass`**: The input data is passed through the network layer by layer to compute the output predictions.
2. **`Loss Calculation`**: The loss function quantifies the difference between the predicted outputs and the actual target values.
3. **`Backward Pass`**: The gradients of the loss function with respect to each weight and bias are computed using the chain rule of calculus. This process is known as backpropagation.
4. **`Weight Update`**: The weights and biases are updated using the computed gradients and a learning rate, which determines the step size for each update.

2.1.4 Neural Network Implementation

For our problem, our goal is to reduce the dimensionality of the input data (femur shapes represented as high-dimensional vectors) to a lower-dimensional representation while preserving as much information as possible.

We tried several architectures but each one had this same characteristics:

- Input layer size: 54873 (number of coordinates of the femur mesh)
- Output layer size: 54873 (reconstructed femur mesh)
- Hidden layers: several layers with decreasing and then increasing sizes to create a bottleneck effect, forcing the network to learn a compressed representation of the input data.
- A latent space size of 10.

To train the network, we used a dataset of femur shapes, splitting it into training and validation sets. During the training process, we gave the network femur shapes as input and used the same shapes as target outputs, effectively training the network to reconstruct the input data.

After the training, we could use the encoder part of the network to obtain a low-dimensional representation of any femur shape, and the decoder part to reconstruct the femur shape from its low-dimensional representation.

2.2 Multi-threading

Since the NN is quiet slow to train, we implemented a multi-threading system to speed up the process.
Firstly we try to create a thread for each operation that can be parallelized. But the overhead created by the creation of threads is too important compared to the time saved (we began with vector addition):

With this approach, the time taken to train our network is too long because of the large number of threads created, we can't train in a reasonable time.

So, we decided to split the operations in a fixed number of threads (depended of the computer threads, basically 4 or 8). Each thread will compute a part of the result vector.

2.3 PCA

3 Results

3.1 Encoder results

The goal of the encoder is to reduce the dimensionality of the femur shapes from 54873 to 10.

Even with this strong reduction, it's still complicated to visualize the results. We decide do a PCA on the latent space to see if we can focuse on fewer dimensions.

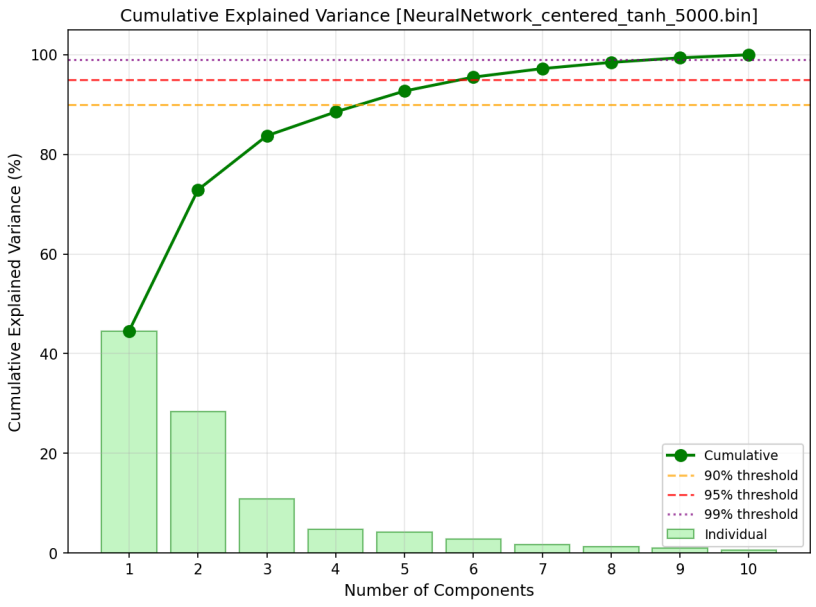


Figure 4 - Cumulative variance explained by PCA on the latent space

We can see that the first three principal components explain almost 85% of the variance in the latent space.

We decide to plot the data in the latent space using only these three components. To see if there is some structure in the data.

We can see that the data is quite well approximated by a plane in this 3D space. That allow us to add details on the visualization of the latent space.

Combinating these two observation that means that we can play with two parameters to explore the latent space.

3.2 Decoder results

[TODO]

4 Discussion

Glossary

Bibliography